Hands-On Lab Walkthrough: Kafka Streams & ksqlDB In Action

Course: Enterprise Real-Time Data Engineering

Session: S07 — Kafka Streams, Stream-Table Duality & ksqlDB

Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)

Materials: fakhr.me/KafkaSpark | Repo: github.com/fakhruldeen/KafkaSpark

1. Lab Architecture & Objectives

In this hands-on lab, you will build an end-to-end real-time stream processing pipeline for the **Nile Retail Group** retail transactions:
1. Validate Kafka Streams core mechanics (Topologies, State Stores, Changelogs, Crash Recovery) using the pure Python simulator (`python/s07_streams_py.py`).
2. Interact with the multi-broker Confluent ksqlDB cluster via the REST API and the interactive ksqlDB CLI.
3. Formulate continuous declarative stream processing queries using `CREATE STREAM` and `CREATE TABLE` .
4. Perform real-time tumbling time-window aggregations (`WINDOW TUMBLING (SIZE 1 HOUR)`).
5. Diagnose and fix a distributed co-partitioning join defect to enrich streaming orders with customer geographic dimensions (`CUSTOMERS`).

2. Prerequisites & Environment Health Check

Navigate to the course lab root directory on your workstation:

```
cd "${LAB_ROOT:-delivery_materials/00_Lab}" 2>/dev/null || cd 00_Lab || cd ../../00_Lab
```

Step 2.1: Verify the Cluster Services

Run the automated infrastructure check:

```
make check
```

Expected Output:

```
kafka1 (9092)    ... PASS
kafka2 (9093)    ... PASS
kafka3 (9094)    ... PASS
schema-registry  ... PASS
connect (8083)   ... PASS
ksqldb (8088)    ... PASS
```

Step 2.2: Verify ksqlDB Server REST Endpoint

Directly query the ksqlDB server info endpoint:

```
curl -s http://localhost:8088/info | jq .
```

Expected Output:

```
{
  "KsqlServerInfo": {
    "version": "8.4.0",
    "kafkaClusterId": "MkU3OEVBNTcwNTJENDM2Qk",
    "ksqlServiceId": "lab_ksql_",
    "serverStatus": "RUNNING"
  }
}
```

---

3. Part 1: Offline Kafka Streams Mechanics (Python Model)

Before interacting with ksqlDB SQL, inspect how Kafka Streams models local state stores, topologies, and changelog replication in code.

Step 3.1: Inspect the Python Simulator

Open and read `python/s07_streams_py.py`:

```
cat python/s07_streams_py.py
```

Notice how:
- `KeyedAggregate`: Represents an embedded state store (like RocksDB) paired with an internal `changelog` list.
- `restore()`: Replays the changelog sequence to rebuild the in-memory state after a simulated crash.
- `run_topology()`: Implements a 3-stage stream processing topology:

$$\text{Filter} \longrightarrow \text{Map} \longrightarrow \text{Aggregate}$$

Step 3.2: Execute the Topology Unit Tests

Run the automated test suite:

```
.venv/bin/pytest -v tests/test_s07_streams_py.py
```

Expected Output:

```
tests/test_s07_streams_py.py::test_topology_filters_and_aggregates PASSED
tests/test_s07_streams_py.py::test_changelog_has_one_entry_per_update PASSED
tests/test_s07_streams_py.py::test_restore_from_changelog_rebuilds_the_same_state PASSED
tests/test_s07_streams_py.py::test_max_aggregate_is_a_different_table PASSED
============================== 4 passed in 0.02s ===============================
```

---

4. Part 2: Interactive ksqlDB Streaming Pipeline

Now execute the production SQL streaming queries on the live multi-broker cluster.

Step 4.1: Launch the Interactive ksqlDB CLI

Open the ksqlDB console client:

```
make ksql
```

(Alternative direct docker command: `docker exec -it ksqldb-server ksql http://localhost:8088`)

You will be greeted by the ksqlDB welcome banner:

```
                  ===========================================
                  =       Enterprise ksqlDB Interactive     =
                  ===========================================
ksql>
```

Step 4.2: Configure Auto Offset Reset to Earliest

Instruct ksqlDB to process all existing historical records on topics:

```
SET 'auto.offset.reset' = 'earliest';
```

Step 4.3: Define the Base Ingestion Stream (`ORDERS_RAW`)

Map the underlying Kafka topic `orders.events.v1` into a typed stream:

```
CREATE STREAM IF NOT EXISTS orders_raw (
  order_id VARCHAR KEY,
  event_id VARCHAR,
  customer_id VARCHAR,
  store_id VARCHAR,
  event_type VARCHAR,
  amount DOUBLE,
  event_time VARCHAR
) WITH (KAFKA_TOPIC='orders.events.v1', VALUE_FORMAT='JSON', PARTITIONS=3);
```

Verify the stream exists:

```
SHOW STREAMS;
```

---

5. Part 3: Continuous Filtering & Hourly Revenue Windows

Step 5.1: Create the Filtered Payments Stream (`CSAS`)

Filter out pending or cancelled orders to keep only valid paid transactions:

```
CREATE STREAM IF NOT EXISTS payments AS
  SELECT * FROM orders_raw
  WHERE event_type = 'PaymentAccepted' AND amount IS NOT NULL
  EMIT CHANGES;
```

Notice: ksqlDB automatically creates an underlying Kafka topic named `PAYMENTS` with 3 partitions and a replication factor of 3.

Step 5.2: Create the Hourly Revenue Tumbling Window Table (`CTAS`)

Aggregate revenue per physical store per 1-hour tumbling window:

```
CREATE TABLE IF NOT EXISTS revenue_by_store_hour AS
  SELECT store_id, SUM(amount) AS revenue, COUNT(*) AS payments
  FROM payments
  WINDOW TUMBLING (SIZE 1 HOUR)
  GROUP BY store_id
  EMIT CHANGES;
```

Inspect the materialized table and its underlying changelog:

```
SHOW TABLES;
DESCRIBE REVENUE_BY_STORE_HOUR EXTENDED;
```

---

6. Part 4: Stream-Table Joins & Co-Partitioning Repair

Step 6.1: Register the Reference Customer Table

Map the delimited CSV topic `customers.v1` into a primary-key table:

```
CREATE TABLE IF NOT EXISTS customers (
  customer_id VARCHAR PRIMARY KEY,
  region VARCHAR,
  updated_at VARCHAR
) WITH (KAFKA_TOPIC='customers.v1', VALUE_FORMAT='DELIMITED', PARTITIONS=3);
```

Step 6.2: Repartition Payments by Customer ID

Because `payments` was keyed by `order_id` while `customers` is keyed by `customer_id`, we must satisfy the **Co-Partitioning** constraint:

```
CREATE STREAM IF NOT EXISTS payments_by_customer AS
  SELECT * FROM payments PARTITION BY customer_id
  EMIT CHANGES;
```

Step 6.3: Execute the Stream-Table Left Join

Enrich payments with customer regional attributes:

```
CREATE STREAM IF NOT EXISTS payments_enriched AS
  SELECT p.customer_id, p.store_id, p.amount, c.region
  FROM payments_by_customer p
  LEFT JOIN customers c ON p.customer_id = c.customer_id
  EMIT CHANGES;
```

---

7. Part 5: Testing Live Event Propagation

Open a second terminal window. While ksqlDB is running, produce a live payment event:

```
cd "${LAB_ROOT:-delivery_materials/00_Lab}" 2>/dev/null || cd 00_Lab || cd ../../00_Lab
.venv/bin/python -c "
from confluent_kafka import Producer
import json

p = Producer({'bootstrap.servers': 'localhost:9092'})
event = {
    'order_id': 'ORD-LIVE-7001',
    'event_id': 'EVT-7001',
    'customer_id': 'C0001',
    'store_id': 'STORE-ALEX-1',
    'event_type': 'PaymentAccepted',
    'amount': 450.0,
    'event_time': '2026-11-06T12:00:00Z'
}
p.produce('orders.events.v1', key='ORD-LIVE-7001', value=json.dumps(event))
p.flush()
print('Live order produced to orders.events.v1')
"
```

Now in your ksqlDB CLI, run a continuous Push Query:

```
SELECT customer_id, store_id, amount FROM payments EMIT CHANGES LIMIT 3;
```

You will immediately observe the live transaction emitted on screen!

---

8. Summary & Automated Verification Gate

To execute the full automated verification test suite for Session S07, run:

```
bash ../S07_Kafka_Streams_and_ksqlDB/Trainer/verify_s07.sh
```

Expected Output:

```
Running S07 Automated Kafka Streams & ksqlDB Verification Gate...
[1/5] Probing ksqlDB REST endpoint (http://localhost:8088)... PASS (HTTP 200 OK & serverStat
us: RUNNING)
[2/5] Running Kafka Streams Python topology unit tests (KeyedAggregate)... PASS (4/4 tests p
assed: StateStore, Topology, Changelog, Restore)
[3/5] Verifying orders.events.v1 and customers.v1 base topics... PASS (Base topics ready wit
h 3 partitions and RF 3)
[4/5] Executing and validating ksqlDB streaming topologies (CSAS / CTAS)... PASS (orders_ra
w, payments, revenue_by_store_hour, customers, payments_enriched active)
[5/5] Testing end-to-end event filtering into PAYMENTS topic... PASS (PaymentAccepted event
successfully processed and routed by ksqlDB)

✔ S07 Kafka Streams & ksqlDB Quality Gate: 100% PASS
```

🌟 Gold Stretch Challenge: Hopping Window Revenue Aggregation in ksqlDB

Distinguished Engineer Challenge: If you complete the core lab before time expires, build a real-time sliding window table:

1. Create a 15-minute hopping window with a 5-minute advance interval in ksqlDB:

```sql
CREATE TABLE store_revenue_hopping AS
SELECT
  store_id,
  COUNT(*) AS total_orders,
  SUM(amount) AS total_revenue
FROM retail_orders_stream
WINDOW HOPPING (SIZE 15 MINUTES, ADVANCE BY 5 MINUTES)
GROUP BY store_id
EMIT CHANGES;
```

2. Query the pull query endpoint to inspect window bounds (`WINDOWSTART` , `WINDOWEND`).